



SÃO PAULO
GOVERNO DO ESTADO

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas - AMS

Gabriel Sales Dorea

Ian Matheus Moura

Jade Nogueira Silva

João Vitor Perez Saraiva

Matheus de Araújo Emídio

Documentação de Desenvolvimento de Software Projeto Corretora de Criptomoedas

Sorocaba
Dezembro - 2025

1. Visão Geral do Projeto

Este projeto interdisciplinar tem como objetivo o desenvolvimento do protótipo de uma plataforma de corretagem de criptoativos inspirada em soluções consolidadas do mercado, como a Binance, mas adaptada a um conceito original: a empresa fictícia **Lunaria**. A Lunaria tem como razão social oferecer um ambiente acessível para atividades econômicas envolvendo criptoativos, prezando por uma comunicação clara, simples e educativa. Seu diferencial está justamente na proposta de atingir não apenas usuários experientes, mas também o público geral, oferecendo uma interface amigável e um ecossistema que facilite o entendimento e o uso cotidiano de ativos digitais.

Para viabilizar essa proposta, o projeto adota uma arquitetura distribuída baseada em microserviços, garantindo escalabilidade, modularização e flexibilidade na evolução de cada componente. A comunicação é realizada por meio de chamadas síncronas via API Gateway (REST) e, quando necessário, por mensageria assíncrona utilizando RabbitMQ.

A solução integra diversas tecnologias modernas: backend em .NET seguindo princípios de Clean Architecture, frontend web em Next.js com Tailwind CSS, aplicativo mobile em React Native e um chatbot de suporte implementado em Flask (Python). Esse conjunto de ferramentas permite demonstrar, de forma prática, como arquiteturas distribuídas podem sustentar serviços financeiros complexos, mas com foco na clareza e na experiência do usuário final.

Assim, este projeto não apenas apresenta a estrutura técnica da Lunaria, mas também reflete sua missão de tornar o universo dos criptoativos mais acessível, educativo e funcional para o dia a dia das pessoas.

2. Arquitetura da Aplicação

2.1 Estrutura Geral

O sistema é composto por **cinco APIs principais** e um **API Gateway**.

API	Função principal
UserAPI	Cadastro, autenticação e controle de acesso dos usuários.

WalletAPI	Controle de carteiras, saldos e transações.
CurrencyAPI	Consulta de cotações e histórico de preços de ativos.
ChatbotAPI	Atendimento automatizado e comandos de interação.
GatewayAPI	Camada intermediária entre o frontend/mobile e os microserviços.

2.2 Componentes de Frontend e Mobile

- **Frontend (Next.js + Tailwind CSS):** interface principal de acesso às funcionalidades, exibindo saldo, ativos, histórico e chat.
- **Mobile (React Native + Expo):** versão simplificada e otimizada para acesso rápido a saldo e transações.

2.3 Comunicação entre Componentes

- **Síncrona (REST/HTTP):** comunicação direta entre frontend e backend via API Gateway.

3. Estrutura de Pastas e Clean Architecture

A arquitetura segue o padrão **Clean Architecture**, garantindo separação de responsabilidades, testabilidade e facilidade de manutenção.

3.1 Estrutura Base de um Microserviço

```

/userAPI
  /API
    /Controllers
    /DTOs
    /Configurations
    Program.cs
  /Domain
    /Entities
    /Interfaces
  /Infrastructure
    /Data
    /Repositories

```

```
/Migrations
/Application
/Services
/Interfaces
/UseCases
```

3.2 Camadas da Arquitetura

- 1. API (Apresentação)** Exposição de endpoints REST. Exemplo: UserController.cs responde às requisições de cadastro e login.
- 2. Domain (Domínio)** Contém as entidades e interfaces base do negócio. Exemplo: User.cs, IUserRepository.cs.
- 3. Infrastructure (Infraestrutura e Persistência)** Gerencia o banco de dados e implementa os repositórios. Exemplo: UserRepository.cs, UserDbContext.cs.
- 4. Application (Casos de Uso e Regras de Negócio)** Implementa a lógica de aplicação, serviços e casos de uso. Exemplo: RegisterUserUseCase.cs, UserService.cs.

4. Autenticação e Segurança

A autenticação é feita com **JWT (JSON Web Token)**, garantindo que apenas usuários autenticados acessem rotas protegidas.

4.1 Dependências utilizadas

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package BCrypt.Net-Next
```

4.2 Fluxo de Autenticação

1. O usuário realiza login enviando e-mail e senha.
2. A API valida as credenciais e gera um **token JWT**.
3. O frontend armazena o token e o envia no cabeçalho das próximas requisições.
4. O token é validado em cada chamada por middleware de autenticação.

4.3 Exemplo de Login

Request

```
POST /user/login
{
  "email": "user@example.com",
  "password": "123456"
}
```

Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IHR5cGU6IjEudmVudCIsImF1dG8iOiJ1b250b290cm9udC5jb20iLCJmfaRequired": true,
  "mfaType": "sms"
}
```

4.4 Hash de Senhas

Na criação do usuário, a senha deve ser armazenada de forma segura:

```
var hashedPassword = BCrypt.Net.BCrypt.HashPassword(userDto.Password);
```

5. Comunicação entre Serviços

Os microserviços se comunicam de duas formas:

5.1 Comunicação Síncrona (REST via GatewayAPI)

Usada em fluxos que exigem resposta imediata.

Exemplos:

- Login (POST /user/login)
- Consulta de saldo (GET /wallet/balance)
- Execução de trade (POST /wallet/trade)
- Envio de mensagem ao chatbot (POST /chatbot/message)

Exemplo de resposta resumida:

```
{  
  "tradeId": "t98765",  
  "status": "SUCCESS",  
  "newBalances": { "BTC": 0.5, "USDT": 13750.20 }  
}
```

5.3 Fluxo de Trade de Ativos

sequenceDiagram

```
participant FE as Frontend  
participant GW as API Gateway  
participant W as API Wallet  
participant C as API Currency  
participant MQ as Broker  
FE->>GW: POST /wallet/trade  
GW->>W: Solicita execução  
W->>C: Consulta preço atual  
C-->>W: Retorna cotação  
W-->>GW: Retorna trade concluído  
Note over W, MQ: Publica evento TRADE_SUCCESS
```

5.4 Fluxo de Depósito via Chatbot

sequenceDiagram

```
participant FE as Frontend  
participant CB as API Chatbot  
participant MQ as Broker  
participant W as API Wallet  
FE->>CB: "Depositar 200 USD"  
CB->>MQ: Publica comando DEPOSIT  
MQ-->>W: Executa depósito  
W->>MQ: Publica WALLET_DEPOSIT_SUCCESS  
MQ-->>CB: Notifica sucesso
```

6. Requisitos Funcionais e Não Funcionais (detalhado)

6.1 Requisitos Funcionais (RF)

Cada RF está numerado para referência nas entregas e testes.

RF-01 — Cadastro de usuário

- Permitir que um usuário crie conta com nome, email e senha.
- Validar unicidade de email.
- Armazenar senha hasheada (BCrypt).

RF-02 — Autenticação

- Login via email + senha.
- Emissão de **JWT** com validade configurável.
- Suporte a MFA (campo mfaRequired no payload). Implementação mínima: simulação com sms flag (não é preciso integrar SMS real; pode ser flag simulada para entrega).

RF-03 — Consulta de saldo

- Exibir saldo total e saldos por carteira (spot, funding, overview).
- Endpoint: GET /wallet/balance?userId={id} (autenticado).

RF-04 — Simulação de depósito/saque

- Executar depósito fictício em carteira selecionada.
- Publicar evento de depósito (wallet.deposit.success) no broker.
- Endpoint: POST /wallet/deposit.

RF-05 — Trade (troca de ativos)

- Permitir troca entre dois ativos (ex: BTC → USDT) com verificação de saldo.
- Consultar preço atual na CurrencyAPI.
- Registrar transação e publicar evento wallet.trade.success.

RF-06 — Exibição de ativos populares e gráfico histórico

- GET /currency/list — listar ativos populares.

- GET /currency/history?symbol={} — retornar históricos (últimas 24h, 7d) para plotagem no frontend.

RF-07 — Chatbot (comandos e consultas)

- POST /chatbot/message — responde consultas (ex: "Qual meu saldo?") e aceita comandos simulados (ex: "Depositar 200 USD").
- Chatbot publica comandos no broker para ações que afetam outras APIs.

RF-08 — Perfil do usuário

- Endpoint para leitura/atualização de dados não sensíveis (nome, preferências).
- Endpoint protegido para upload de avatar (opcional, armazenar referência).

RF-09 — Histórico e extrato

- GET /wallet/history?userId={} — lista de transações (depósitos, saques, trades).

RF-10 — Logs e auditoria (mínimo)

- Registrar operações sensíveis (login, trade, depósito) em log acessível para avaliação (arquivo ou tabela simples).

6.2 Requisitos Não Funcionais (RNF)

RNF-01 — Desempenho

- Tempo de resposta médio para endpoints principais (/wallet/balance, /currency/price) $\leq 1s$ em ambiente de testes local/avaliativo.

RNF-02 — Segurança

- Senhas armazenadas com bcrypt.
- JWT com assinatura HMAC (ou RSA se preferirem) e expiração.
- Rotas protegidas por middleware de autenticação.
- Validação de entrada (sanitização) em todos os endpoints.

RNF-03 — Escalabilidade

- Arquitetura baseada em microserviços; componentes desacoplados via RabbitMQ.
- Configurações para permitir execução em contêineres (Docker) — containers separados por serviço.

RNF-04 — Manutenibilidade

- Padrão de camadas (Clean Architecture).
- Documentação mínima por serviço: README com endpoints, dependências e rota de execução local.

RNF-05 — Observabilidade

- Logs estruturados (pelo menos console + arquivo).
- Métricas simples (contadores de requisições e latência) ou prints no ambiente de avaliação.

RNF-06 — Portabilidade / Deploy

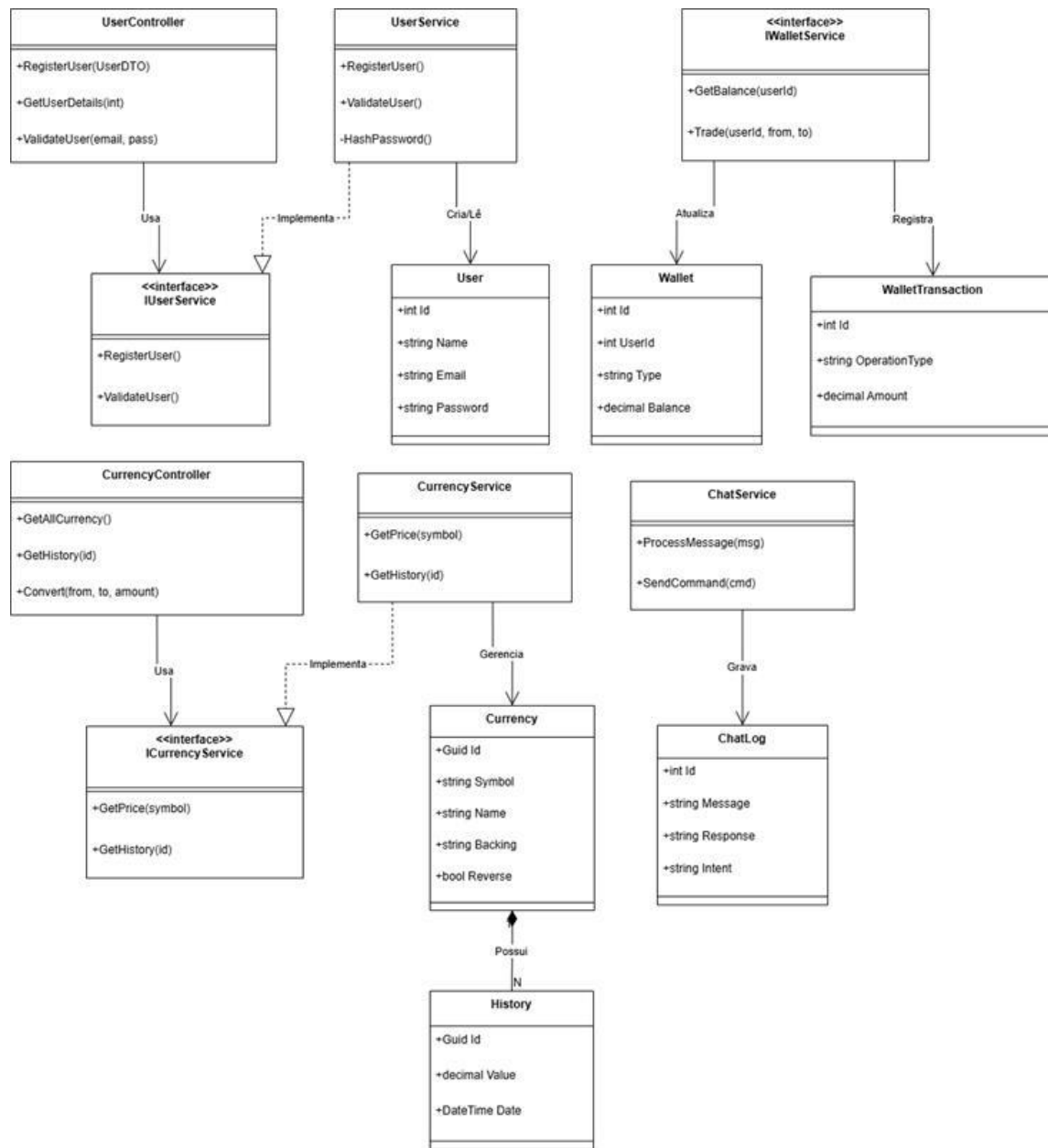
- Aplicações devem rodar localmente com dotnet run (APIs) e npm run dev (frontend) e, preferivelmente, via Docker Compose para avaliação integrada.

RNF-07 — Tolerância a falhas

- Em caso de falha de um consumidor RabbitMQ, eventos devem ser reencaminhados / mantidos na fila (configuração básica de retry/ack).

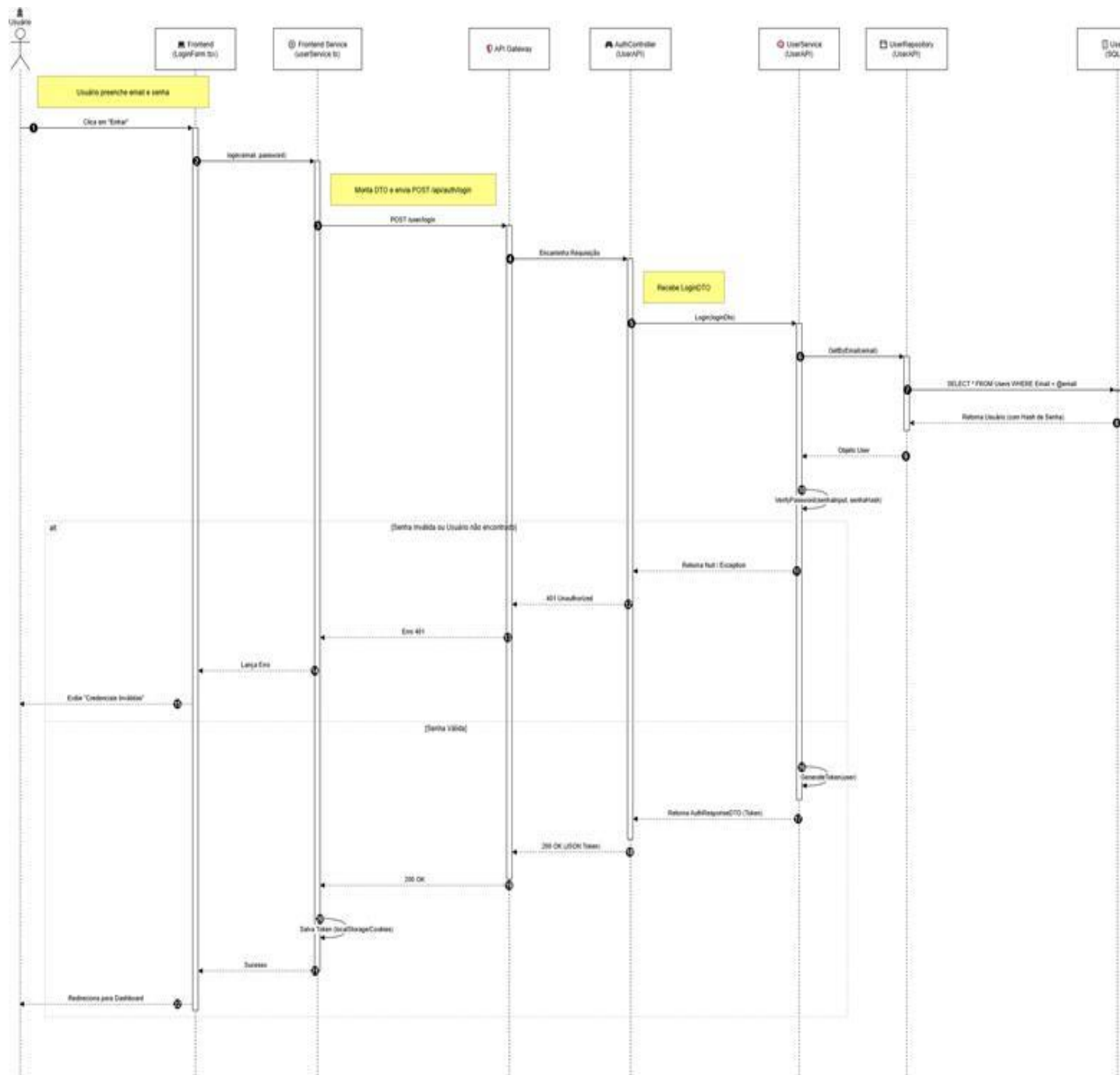
6.3 Diagramas de dados

Figura 01 – Diagrama de classes



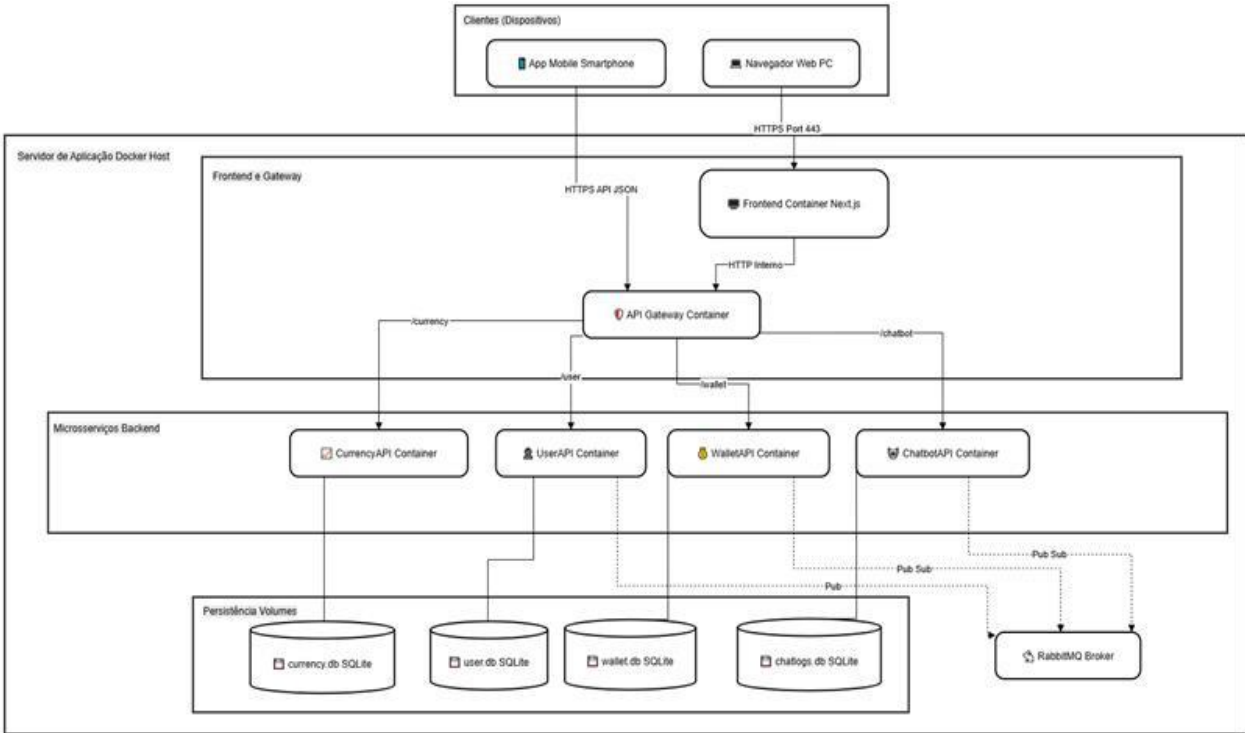
Fonte: Autoria própria

Figura 02 – Diagrama de sequência



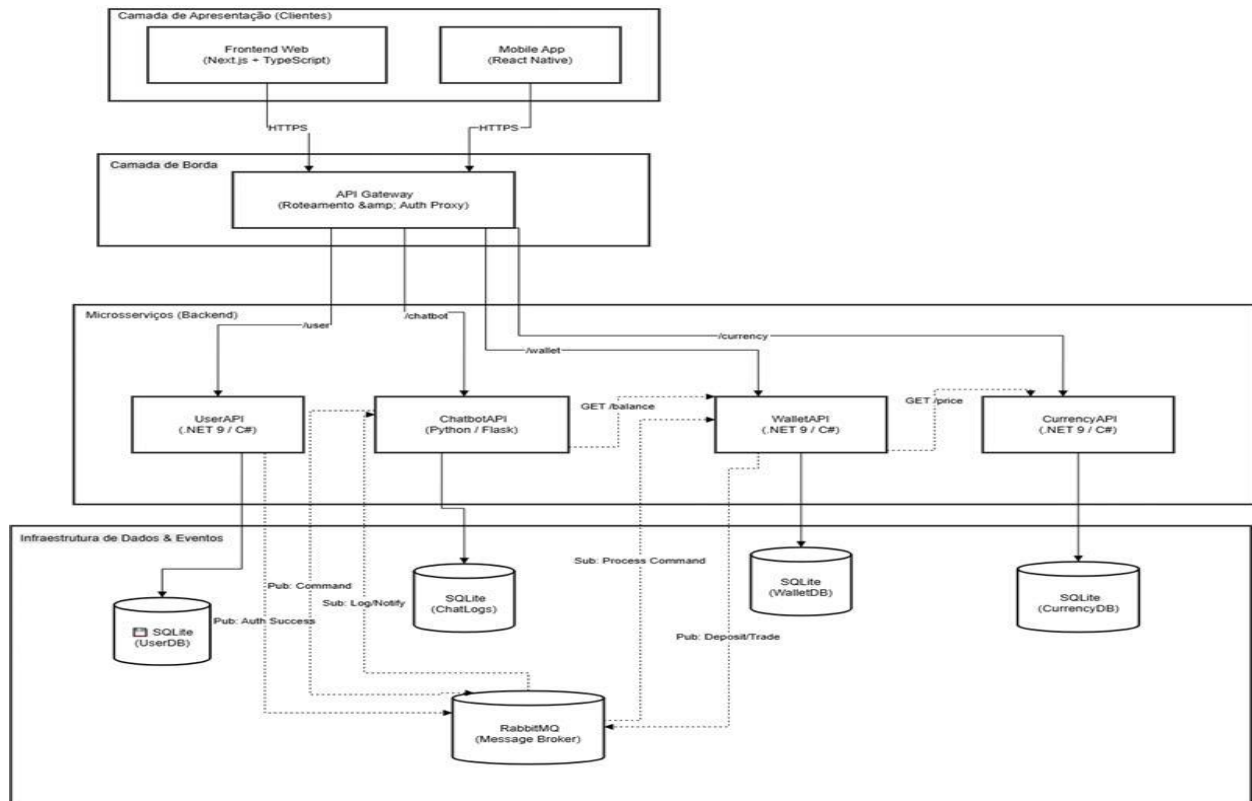
Fonte: Autoria própria

Figura 03 – Diagrama de implantação



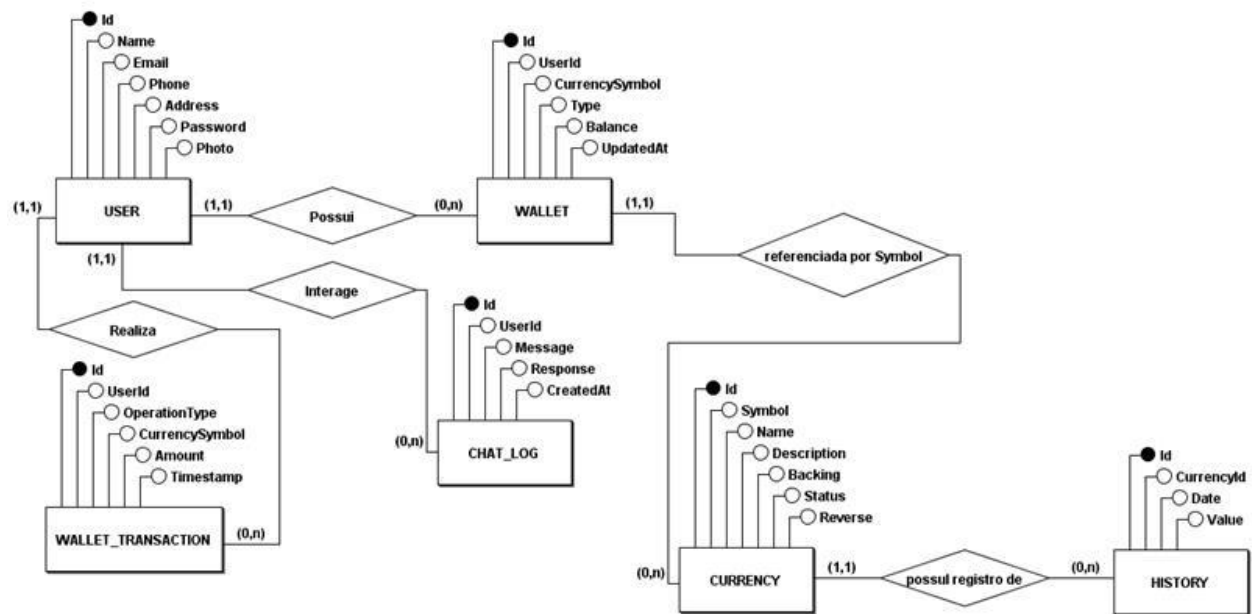
Fonte: Autoria Própria

Figura 04 – Diagrama de arquitetura



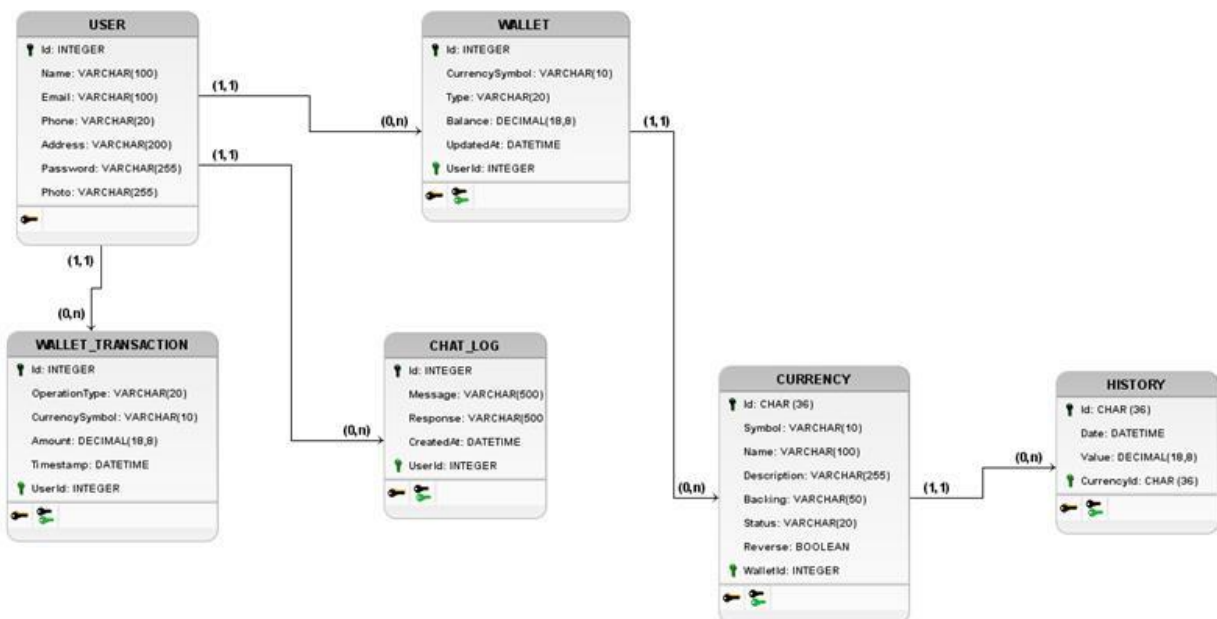
Fonte: Autoria própria

Figura 05 – Modelo Lógico



Fonte: Autoria Própria

Figura 06 – Modelo Lógico



Fonte: Autoria Própria

6.4 Modelos de Dados (resumo mínimo)

User

```

{
  "id": "u123",
  "name": "Matheus Emídio",
  "email": "Emídio@gmail.com",
  "passwordHash": "<bcrypt>",
  "createdAt": "2025-09-01T12:00:00Z"
}
  
```

WalletBalance

```

{
  "userId": "u123",
  "totalBalance": 15200.50,
  "currency": "USD",
  "wallets": [
    { "type": "spot", "balance": 8500.00 },
    { "type": "funding", "balance": 4700.50 },
  ]
}
  
```

```
{ "type": "overview", "balance": 2000.00 }  
]  
}
```

Trade

```
{  
  "tradeId": "t98765",  
  "userId": "u123",  
  "fromAsset": "BTC",  
  "toAsset": "USDT",  
  "amount": 1.0,  
  "executedPrice": 27500.40,  
  "status": "SUCCESS",  
  "timestamp": "2025-09-19T15:05:00Z"  
}
```

7. Conclusão

O desenvolvimento da aplicação de corretora de criptomoedas permitiu integrar, de forma prática, os principais conceitos estudados ao longo do semestre em sistemas distribuídos, arquitetura de software e engenharia de aplicações. A solução proposta demonstrou como microserviços, comunicação via API Gateway e mensageria com RabbitMQ podem atuar conjuntamente para fornecer um sistema robusto, escalável e modular.

Assim, o projeto cumpre seu propósito interdisciplinar ao simular uma plataforma semelhante às grandes corretoras do mercado, oferecendo uma visão abrangente sobre o ciclo de desenvolvimento, desde a modelagem dos dados até a comunicação entre serviços. O trabalho reforça a importância da modularização, da documentação e da integração entre diferentes tecnologias, preparando a equipe para desafios reais no ambiente profissional.

